

SupportAI: AI-Powered Customer Support Automation

A Salesforce Service Cloud & Agentforce Simulation Platform

Project Description:

This project simulates a Salesforce Service Cloud environment enhanced with an Agentforce-style AI layer for AeroGadgets Global, a fictional consumer electronics brand. The console gives support agents a unified workspace to manage cases, converse with an AI assistant grounded in case context, design reusable Einstein-style prompt templates, and reference technical documentation — all in a single-page application.

The interface is organized into four tabs: a CRM Service Console for case queues and case detail, an Agentforce AI Assistant for grounded conversational support, an Einstein Prompt Template Builder for designing and previewing reusable prompts, and a Technical Documentation Hub. Under the hood, a React 19 + Vite frontend talks to an Express.js API layer (server.ts), which calls the Google Gemini API (gemini-3.5-flash) to generate grounded, case-aware responses. Secure key management is handled via environment variables, and the app was scaffolded from Google AI Studio's app template and published to Vercel.

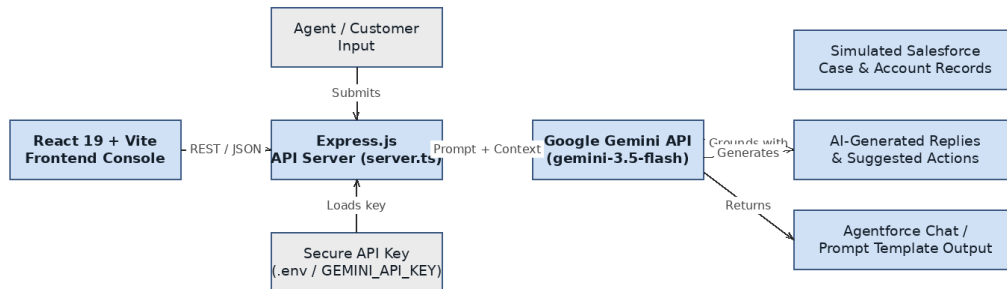
Scenarios

Scenario 1: A support agent opens a high-priority case about a drone that won't power on after a firmware update. The Agentforce AI Assistant summarizes the case, pulls in the relevant knowledge article, and drafts a customer-facing reply recommending a firmware rollback, all grounded in the case's actual details.

Scenario 2: An agent uses the Einstein Prompt Template Builder to create a reusable “Case Summary Generator” template with merge fields for subject, description, priority, and account name. The live preview panel shows exactly how the template renders against a real case before it's deployed to the console.

Scenario 3: A new team member opens the Technical Documentation Hub to understand how the frontend, backend, and Gemini API fit together before making changes, using the architecture section as a reference instead of asking a teammate.

Technical Architecture:



Description: The application uses a three-tier architecture. The React 19 + Vite frontend renders the four-tab console and collects agent input. It sends REST/JSON requests to an Express.js API layer (server.ts), which builds a grounded prompt from the simulated Salesforce case/account records and calls the Google Gemini API (gemini-3.5-flash) to generate the reply, suggested action, or prompt-template output. The Gemini API key is loaded securely from environment variables and never exposed to the frontend.

(Note: This project simulates Salesforce records in-app rather than connecting to a live Salesforce org; the same architecture would extend to a real org via Salesforce REST/Apex callouts.)

Pre-requisites:

- Node.js & npm knowledge: for installing dependencies and running the Vite dev server
- Google Gemini API familiarity: for generating an API key and understanding model parameters
- React & TypeScript skills: for the frontend components and typed API contracts
- Express.js knowledge: for the API routes defined in server.ts
- Salesforce Service Cloud & Agentforce concepts: cases, accounts, prompt templates
- Version control with Git: for cloning and managing the repository
- Vercel familiarity: for public deployment

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Generate a Gemini API key and configure it securely in the project environment.
- Activity 1.2: Research and select the appropriate Gemini model for grounded, case-aware responses.
- Activity 1.3: Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- Activity 1.4: Set up the development environment, installing necessary libraries and dependencies for Vite, React, and Express.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop the core functionalities: Case Console, Agentforce Chat, and Prompt Template Builder.
- Activity 2.2: Implement the Express backend to manage routing and grounded prompt construction.

Milestone 3: server.ts Development

- Activity 3.1: Write the main application logic in server.ts, establishing routes for each feature and integrating Gemini responses.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the CRM Service Console and Agentforce AI Assistant tabs.
- Activity 4.2: Design and develop the Einstein Prompt Template Builder and Technical Documentation Hub tabs.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the server environment.
- Activity 5.2: Deploy the application publicly via Vercel.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate Gemini model for AeroGadgets Global's support use case. This involves reviewing model capabilities, grounding behavior, and response speed, ensuring the chosen model aligns with generating accurate, case-aware replies, summaries, and prompt-template output.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with Gemini, you need a valid API key.

- Step 1 — Create a Google AI Studio account: Visit <https://aistudio.google.com> and sign in with your Google account.
- Step 2 — Navigate to “Get API key”: Generate a new key for the project.
- Step 3 — Add the key to your project: Create a .env file in the project root:

```
GEMINI_API_KEY=your_copied_api_key_here
```

- Step 4 — Verify the key is loaded: server.ts loads this key automatically at startup via dotenv and the @google/genai client.

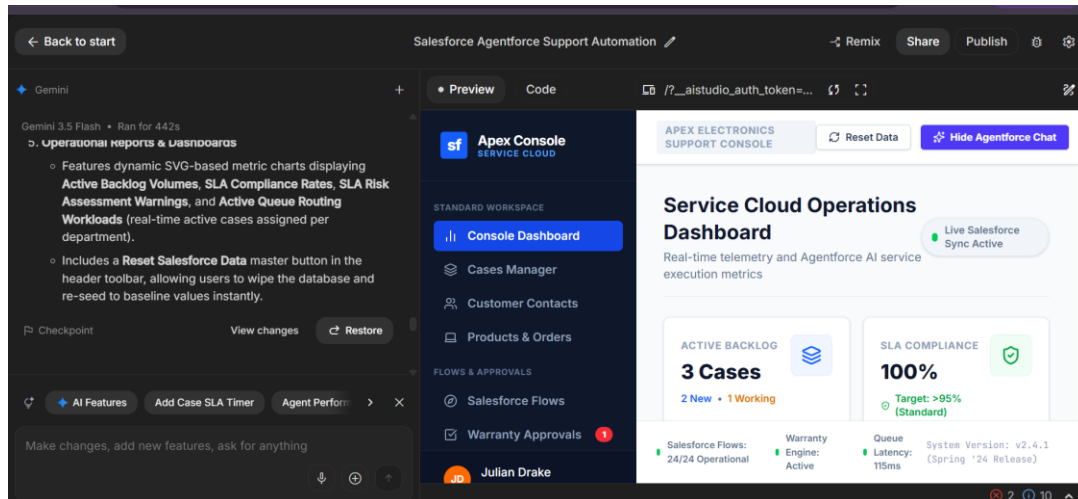
Important — Never commit your API key. Add .env to .gitignore to prevent accidentally pushing it to a public repository.

Activity 1.2: Research and Select the Appropriate Model

- Understand the requirements: review the console's needs — case summaries, grounded chat replies, and prompt-template previews.
- Explore Gemini's model documentation: compare context window, grounding support, and latency.
- Evaluate performance: weigh response quality and speed for interactive, chat-style use.
- **Select the optimal model:** choose gemini-3.5-flash for its balance of speed and quality, with temperature 0.4 for consistent, grounded output.

Activity 1.3: Define the Architecture of the Application

- Draft an architectural diagram covering the frontend (React 19 + Vite), backend (Express.js), and Gemini integration.
- Detail frontend functionality: how agents browse cases, chat with Agentforce, and build prompt templates across four tabs.
- Outline backend responsibilities: how server.ts handles incoming POST requests, builds grounded prompts from simulated case data, and calls the Gemini API.
- Describe AI integration points: how the backend assembles system instructions and case context, calls ai.models.generateContent(), and returns structured results to the frontend.
- The app was scaffolded and iterated on inside Google AI Studio's build environment, which pairs a live chat-driven editor with an instant preview of the running console:



Google AI Studio — build console with live preview of the Service Cloud Operations Dashboard

Activity 1.4: Set Up the Development Environment

- Install Node.js and npm: ensure Node 18+ is installed for Vite and Express compatibility.
- Clone the repository and install dependencies:

```
git clone https://github.com/mrohitsuvaivenkat/ai-powered-customer-support-automation-salesforce
cd ai-powered-customer-support-automation-salesforce && npm install
```

- Configure the API key: create a .env file in the project root with GEMINI_API_KEY.
- Clone the repository, install dependencies, and create the .env file with the Gemini API key:

```

Command Prompt - clone & configure

D:\projects>git clone https://github.com/mrohitsuvaivenkat/
ai-powered-customer-support-automation-salesforce
Cloning into 'ai-powered-customer-support-automation-salesforce'...
Receiving objects: 100% done.

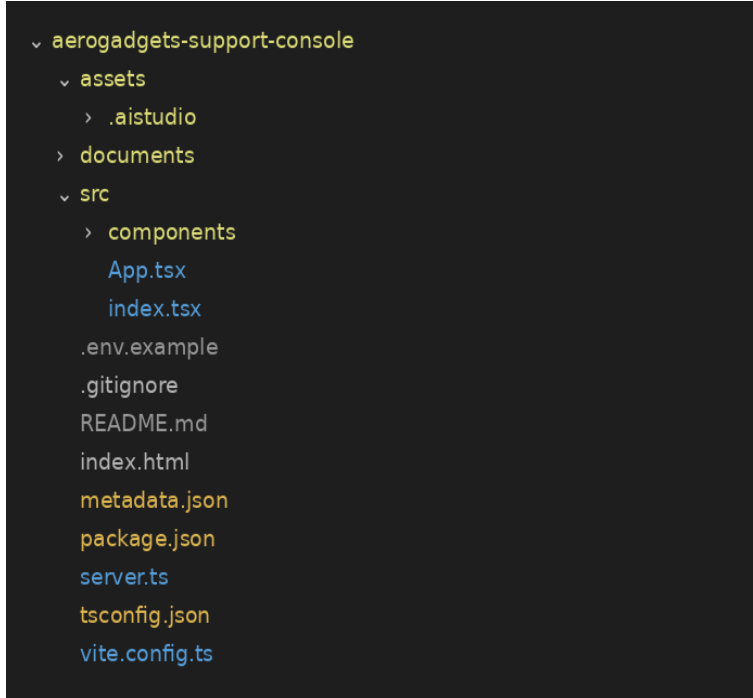
D:\projects>cd ai-powered-customer-support-automation-salesforce
D:\projects>npm install
added 214 packages in 11.8s

D:\projects>notepad .env
GEMINI_API_KEY=your_copied_api_key_here

D:\projects>echo .env >> .gitignore
  
```

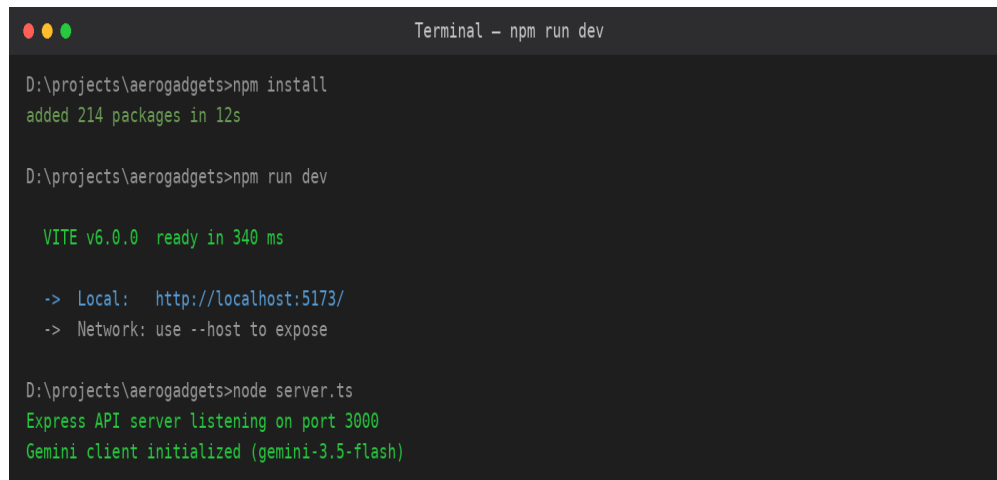
Terminal — cloning the repo and configuring .env

- Review the application structure: the project includes a src folder for components, a documents folder for project docs, and root-level config files (package.json, tsconfig.json, vite.config.ts, server.ts).



Project folder structure (VS Code Explorer)

- Run the development server and confirm both the Vite frontend and Express API start correctly:



```

Terminal - npm run dev

D:\projects\erogadgets>npm install
added 214 packages in 12s

D:\projects\erogadgets>npm run dev

VITE v6.0.0 ready in 340 ms

-> Local: http://localhost:5173/
-> Network: use --host to expose

D:\projects\erogadgets>node server.ts
Express API server listening on port 3000
Gemini client initialized (gemini-3.5-flash)

```

Local development server — npm run dev

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the console's three AI-touching features — case chat, prompt templates, and grounded summaries — as coordinated calls to a single Gemini backend.

- **Agentforce AI Assistant:** generates grounded chat replies using the active case's subject, description, and priority as context.
- **Einstein Prompt Template Builder:** lets agents define reusable prompt templates with merge fields (e.g. `{{!Case.Subject}}`) and preview live output.
- **CRM Service Console:** surfaces the AI's suggested next action directly inside the case detail panel.

Activity 2.2: Implement the Express Backend

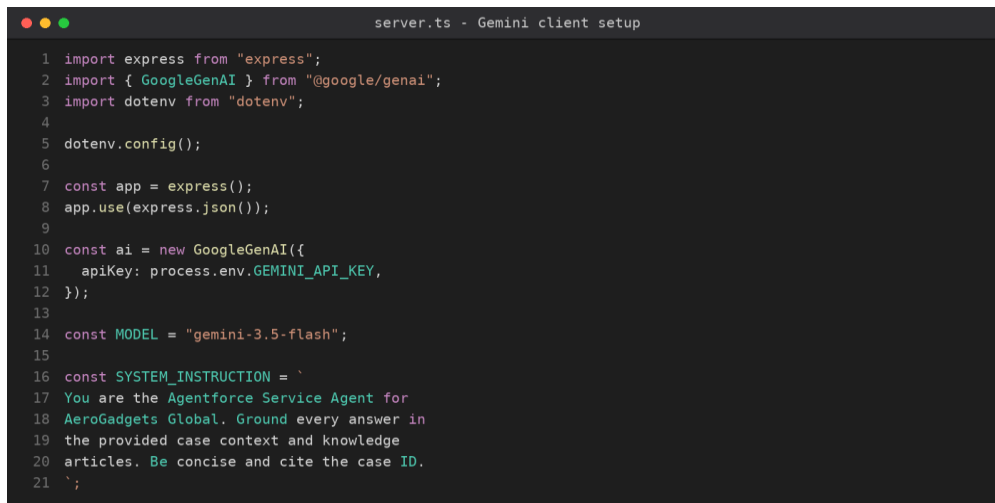
- Define routes in `server.ts` for the chat endpoint, prompt-preview endpoint, and static frontend serving.
- Process user input: accept JSON bodies containing the message, case context, and conversation history.
- Validate that the message is non-empty before calling Gemini.
- Within the chat route, construct a system instruction combining AeroGadgets Global's support policy with the active case's fields, then call `ai.models.generateContent()` with the assembled contents.
- Parse the response text and return a structured JSON payload (success, reply, `caseId`) to the frontend.

Milestone 3: server.ts Development

Milestone 3 focuses on the core logic of `server.ts`, the backbone of the console's backend. This includes setting up the Gemini client, defining the chat route, and ensuring grounded, structured responses.

Activity 3.1: Writing the Main Application Logic in `server.ts`

Initialize Express, load environment variables, and configure the Gemini client with a system instruction scoped to AeroGadgets Global's support policy:



```
server.ts - Gemini client setup

1 import express from "express";
2 import { GoogleGenAI } from "@google/genai";
3 import dotenv from "dotenv";
4
5 dotenv.config();
6
7 const app = express();
8 app.use(express.json());
9
10 const ai = new GoogleGenAI({
11   apiKey: process.env.GEMINI_API_KEY,
12 });
13
14 const MODEL = "gemini-3.5-flash";
15
16 const SYSTEM_INSTRUCTION = `
17 You are the Agentforce Service Agent for
18 AeroGadgets Global. Ground every answer in
19 the provided case context and knowledge
20 articles. Be concise and cite the case ID.
21 `;
```

server.ts — Gemini client setup

Define the `/api/agentforce/chat` route, which validates input, assembles conversation history and case context, calls the Gemini API, and returns the grounded reply:


```
server.ts - /api/agentforce/chat route

1 app.post("/api/agentforce/chat", async (req, res) => {
2   const { message, caseContext, history } = req.body;
3
4   if (!message || typeof message !== "string") {
5     return res.status(400).json({ error: "message is required" });
6   }
7
8   try {
9     const contents = [
10      ...history,
11      { role: "user", parts: [{ text: message }] },
12    ];
13
14     const response = await ai.models.generateContent({
15       model: MODEL,
16       contents,
17       config: {
18         systemInstruction: SYSTEM_INSTRUCTION,
19         temperature: 0.4,
20       },
21     });
22
23     const reply = response.text;
24     res.json({ success: true, reply, caseId: caseContext?.id });
25   } catch (err) {
26     console.error("agentforce/chat failed:", err);
27     res.status(500).json({ error: "Generation failed" });
28   }
29 });
```

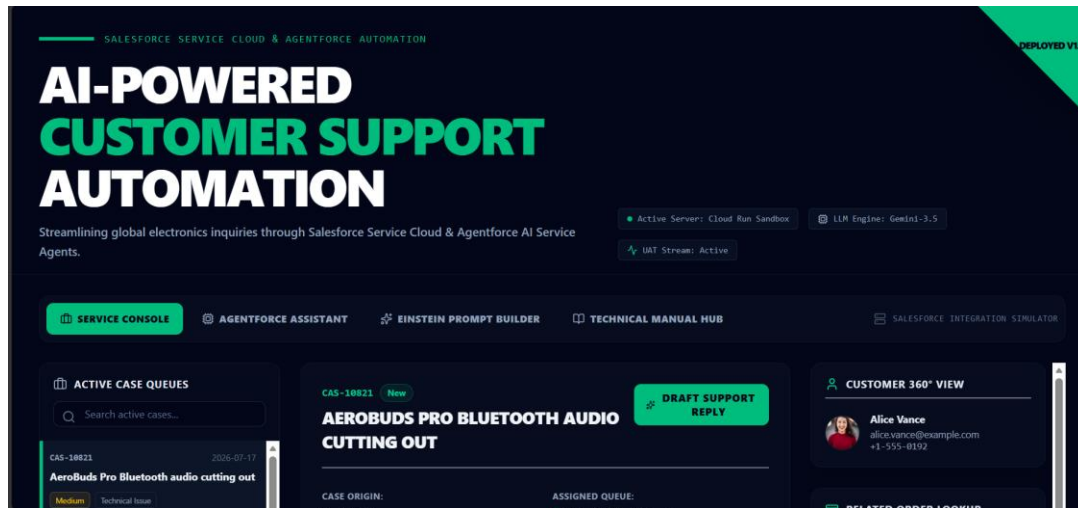
server.ts — /api/agentforce/chat route

Milestone 4: Frontend Development

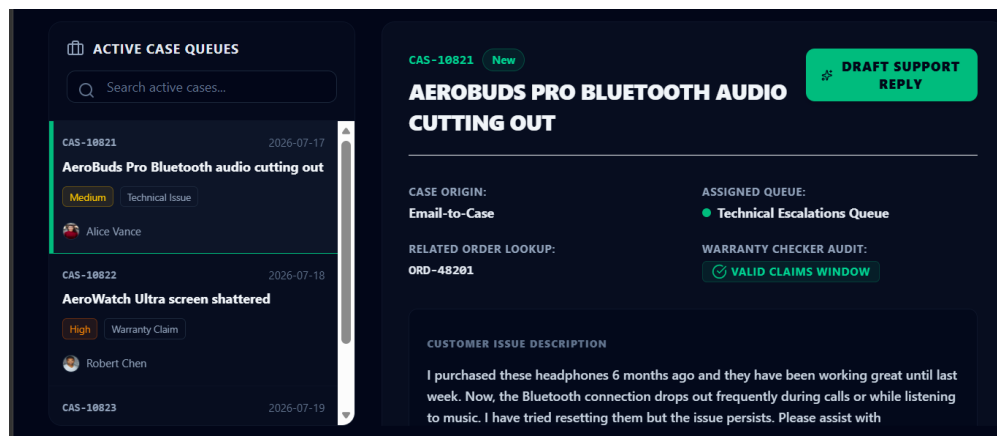
Milestone 4 focuses on building a clean, console-style interface for AeroGadgets Global's support team across four tabs, with React 19 function components and Vite's fast refresh for iteration.

Activity 4.1: CRM Service Console & Agentforce AI Assistant

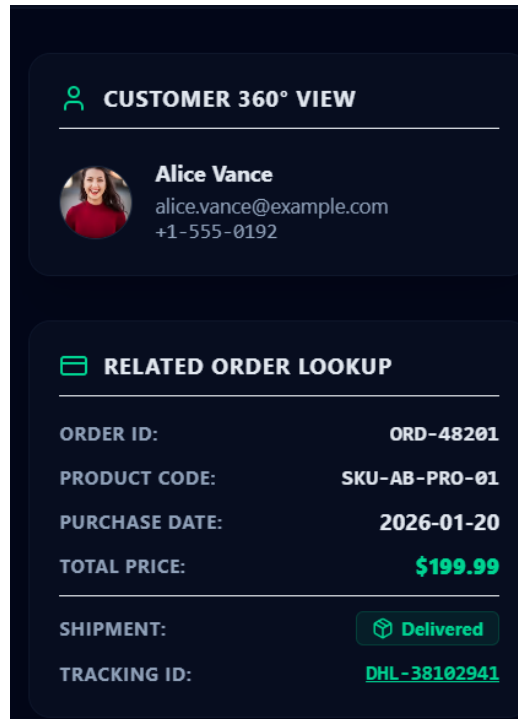
- Build the CRM Service Console tab: a case queue list on the left and a case detail panel on the right, including the AI's suggested next action.



Live app — landing view with Service Console active

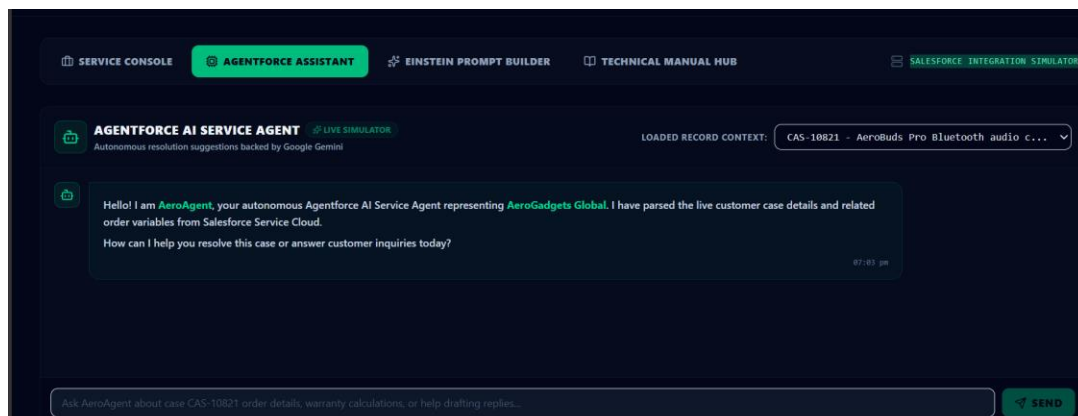


Active Case Queues and case detail, zoomed



Customer 360° View and Related Order Lookup panel

- Build the Agentforce AI Assistant tab: a chat panel grounded in the active case, plus a side panel showing exactly what context (case, account, KB article) is being sent to the model.



Agentforce AI Assistant — live chat grounded in case CAS-10821

Activity 4.2: Einstein Prompt Template Builder & Documentation Hub

- Build the Einstein Prompt Template Builder tab: an editor for the prompt body and merge fields, plus a live preview panel.

Streamlining global electronics inquiries through Salesforce Service Cloud & Agentforce AI Service Agents.

Active Server: Cloud Run Sandbox LLM Engine: Gemini-3.5
UAT Stream: Active

SERVICE CONSOLE AGENTFORCE ASSISTANT **EINSTEIN PROMPT BUILDER** TECHNICAL MANUAL HUB SALESFORCE INTEGRATION SIMULATOR

EINSTEIN PROMPT BUILDER WORKSTATION
Create standard template instruction nodes and bind dynamic Salesforce merge parameters.

ACTIVE CONTEXT: CAS-18821 CONTACT: Alice Vance ORDER: ORD-48281 WARRANTY: VALID

TEMPLATE INSTRUCTIONS (EDIT RAW TEXT BELOW)

EINSTEIN GENERATED RESULT

READY TO DRAFT
Click "Preview Grounded Draft" to execute the template.
The system will merge variables from standard objects and construct responses automatically.

Einstein Prompt Builder Workstation — template instructions and active record context

- Build the Technical Documentation Hub tab: a section list and content on the right.

Agents

SERVICE CONSOLE AGENTFORCE ASSISTANT EINSTEIN PROMPT BUILDER **TECHNICAL MANUAL HUB** SALESFORCE INTEGRATION SIMULATOR

ACADEMIC PORTAL CONFIG

SUBMISSION DATE
27 June 2026

TEAM ID CODE

PROJECT NAME

FUNCTIONAL SCOPE DOCUMENT
Academic Step 1 Report Chapter

Copy Full Markdown **PRINT / SAVE PDF**

Technical Manual Hub — configuration and functional scope document

FUNCTIONAL SCOPE DOCUMENT
Academic Step 1 Report Chapter

Copy Full Markdown **PRINT / SAVE PDF**

Team ID	
Project Name	AI-Powered Customer Support & Service Automation
Marks	4 Marks

Technical Manual Hub — Functional Scope Document detail

Milestone 5: Deployment

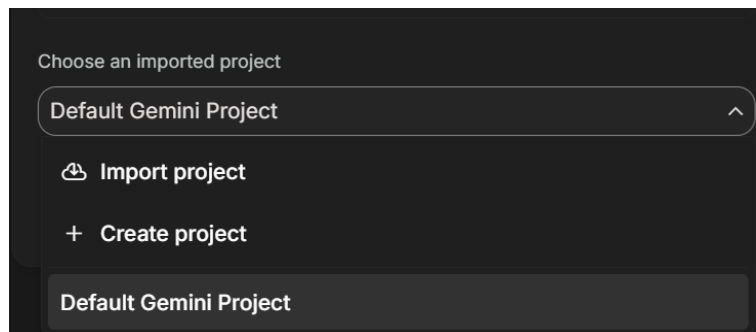
In Milestone 5, the focus is on deploying the console first locally to verify correct operation, and then publicly via Vercel.

Activity 5.1: Preparing the Application for Local Deployment

- Install dependencies with `npm install`.
- Configure the `.env` file with `GEMINI_API_KEY`.
- Run `npm run dev` to start the Vite frontend and Express API together.

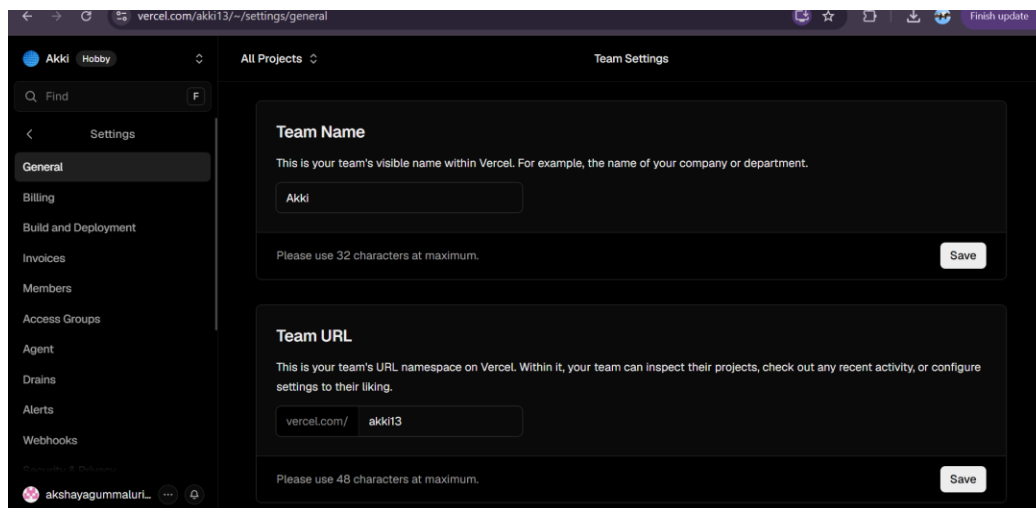
Activity 5.2: Public Deployment via Vercel

- Connect the GitHub repository to a new Vercel project:



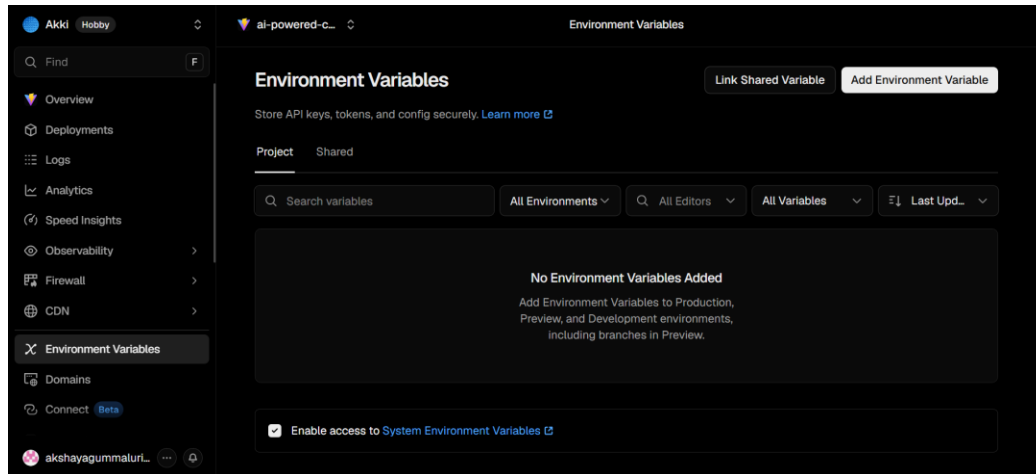
Vercel — choosing the imported project

- Configure team and project settings:



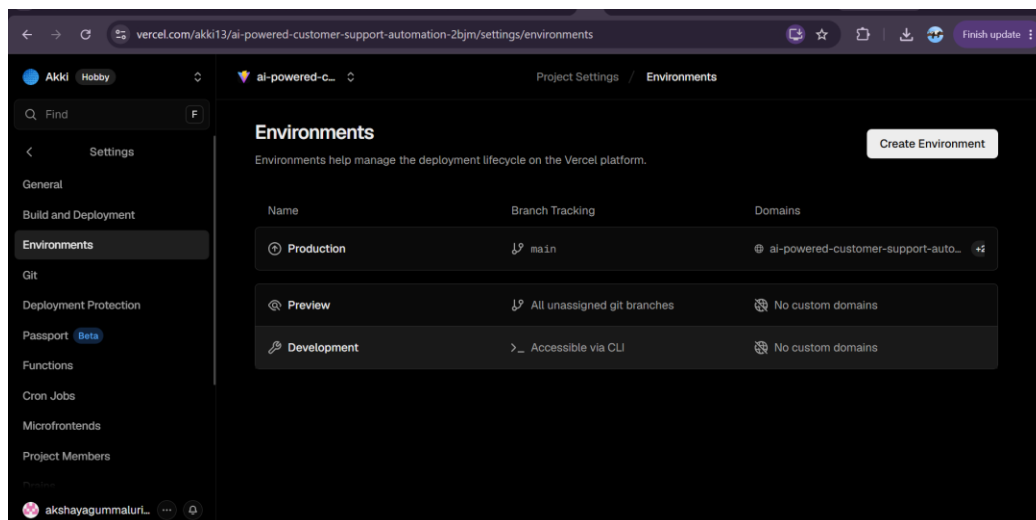
Vercel — Team Settings

- Set the `GEMINI_API_KEY` environment variable in the Vercel project settings:



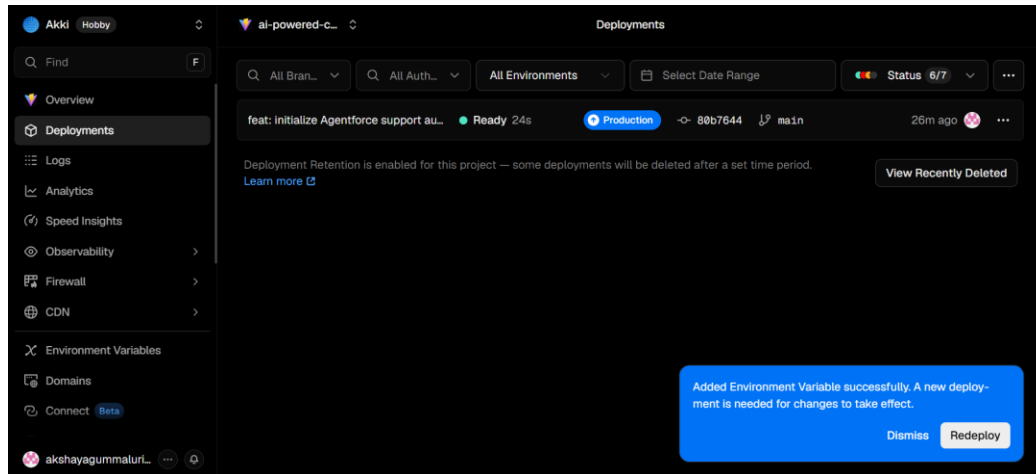
Vercel — Environment Variables

- Review environments (Production, Preview, Development):



Vercel — Environments

- Deploy — Vercel builds the Vite frontend and serves the Express API as serverless functions:



Vercel — successful production deployment

The app is published at:

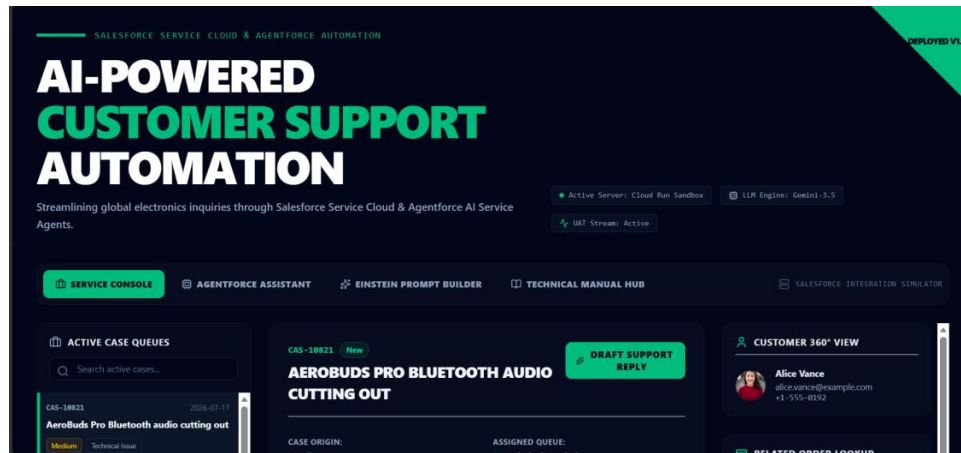
`https://ai-powered-customer-support-automat-ruddy.vercel.app/`

Exploring the Website's Web Pages:

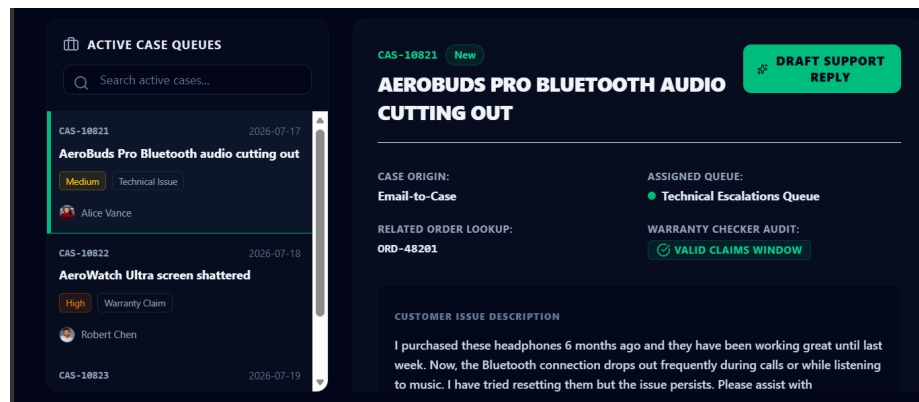
Live app: <https://ai-powered-customer-support-automat-ruddy.vercel.app/>

CRM Service Console Tab

The default landing tab. Agents see a list of open cases on the left and full case detail — including the Customer 360° view and related order lookup — on the right.



Landing view — Service Console active



Active Case Queues and case detail, zoomed


CUSTOMER 360° VIEW


Alice Vance
 alice.vance@example.com
 +1-555-0192


RELATED ORDER LOOKUP

ORDER ID:	ORD-48201
PRODUCT CODE:	SKU-AB-PRO-01
PURCHASE DATE:	2026-01-20
TOTAL PRICE:	\$199.99
SHIPMENT:	 Delivered
TRACKING ID:	DHL-38102941

Customer 360° View and Related Order Lookup

Agentforce AI Assistant Tab

A conversational interface grounded in the currently loaded case record. The record context selector at the top keeps the AI's answers tied to the active case.

SERVICE CONSOLE

AGENTFORCE ASSISTANT

EINSTEIN PROMPT BUILDER

TECHNICAL MANUAL HUB

SALESFORCE INTEGRATION SIMULATOR


AGENTFORCE AI SERVICE AGENT

LIVE SIMULATOR

Autonomous resolution suggestions backed by Google Gemini

LOADED RECORD CONTEXT:

CAS-10821 - AeroBuds Pro Bluetooth audio c...



Hello! I am **AeroAgent**, your autonomous Agentforce AI Service Agent representing **AeroGadgets Global**. I have parsed the live customer case details and related order variables from Salesforce Service Cloud.
 How can I help you resolve this case or answer customer inquiries today?

07:03 pm

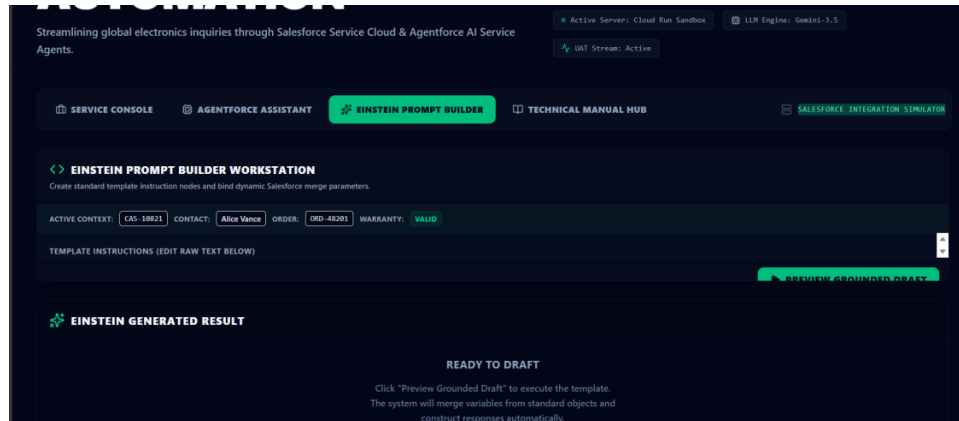
Ask AeroAgent about case CAS-10821 order details, warranty calculations, or help drafting replies.

SEND

Agentforce AI Assistant — live chat grounded in case CAS-10821

Einstein Prompt Template Builder Tab

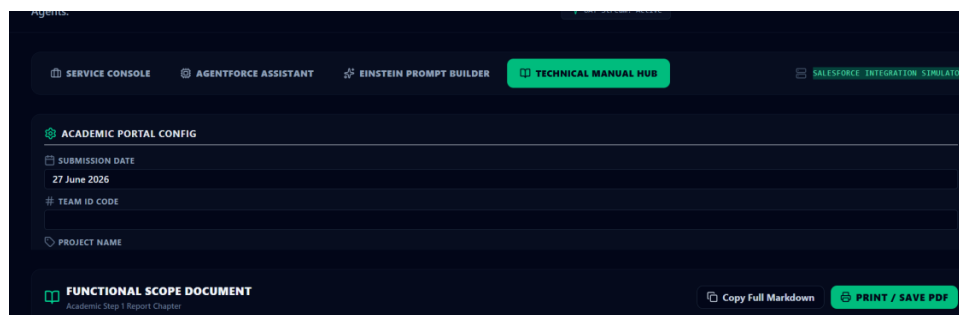
Lets agents define reusable prompt templates bound to live Salesforce-style context (case, contact, order, warranty status) and preview the grounded draft before sending it.



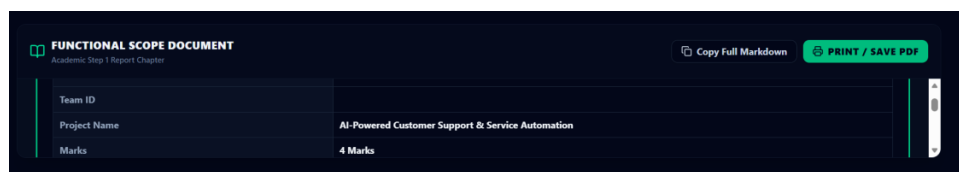
Einstein Prompt Builder Workstation

Technical Documentation Hub Tab

A reference section covering project configuration and generated documentation — useful for onboarding new contributors and for exporting the project's own report.



Technical Manual Hub — configuration panel



Technical Manual Hub — Functional Scope Document detail

Conclusion:

This project delivers a Salesforce Service Cloud and Agentforce-style simulator for AeroGadgets Global, built with a React 19 + Vite frontend, an Express.js backend, and Google's Gemini API for grounded generation. Across four tabs — CRM Service Console, Agentforce AI Assistant, Einstein Prompt Template Builder, and Technical Documentation Hub — the app demonstrates how a support team could summarize cases, chat with a grounded AI assistant, design reusable prompt templates, and reference technical documentation in one place. The project, which included model selection, backend and frontend development, and deployment via Vercel, shows how targeted AI grounded in CRM-style data can meaningfully speed up support workflows. Looking ahead, the project could be extended by connecting to a live Salesforce org via REST/Apex callouts, adding authentication and role-based access, and expanding the knowledge base with real product documentation.